

Design and Implementation of an FPGA-Based Tiled Matrix Multiplication Accelerator for Transformer Self-Attention on the Xilinx KV260 SoM

Richie Li*

University of California, Irvine
Irvine, United States
zhaoqil3@uci.edu

Sicheng Chen

University of California, Irvine
Irvine, United States
sichenc5@uci.edu

ABSTRACT

Transformer-based large language models (LLMs) rely heavily on intensive matrix multiplications for attention and feed-forward layers, with the Q, K, and V linear projections in the Multi-Head Self-Attention (MHA) module constituting a decisive performance bottleneck. In this work, we introduce a highly optimized tiled matrix multiplication accelerator on a resource-constrained Xilinx KV260 FPGA that not only addresses this challenge but sets a new standard for efficiency and performance. Our design exploits persistent on-chip storage, a robust two-level tiling strategy for maximal data reuse, and a systolic-like unrolled compute engine that together deliver unparalleled speed and energy efficiency. Integrated with DistilBERT for Q, K, and V projections, our accelerator achieves an unequivocal $7\times$ speedup over ARM CPU implementations (PyTorch) and an extraordinary $200\times$ improvement over naive NumPy, reaching a throughput of up to 3.1 GFLOPs for matrix multiplications on $(64,768) \times (768,3072)$ matrices while operating at a conservative 100 MHz. These results decisively demonstrate the transformative potential of FPGA-based acceleration for critical Transformer operations, paving the way for scalable and energy-efficient deep learning inference on edge devices.

CCS CONCEPTS

• **Hardware** → **Field-programmable gate arrays**; *Data-flow architectures*.

KEYWORDS

FPGA, matrix multiplication, transformer, self-attention

ACM Reference Format:

Richie Li and Sicheng Chen. 2025. Design and Implementation of an FPGA-Based Tiled Matrix Multiplication Accelerator for Transformer Self-Attention on the Xilinx KV260 SoM. In *Proceedings of ACM Conference (Conference'17)*. ACM, New York, NY, USA, 7 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

*Legal name: Zhaoqin Li

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

Conference'17, July 2017, Washington, DC, USA

© 2025 Association for Computing Machinery.

ACM ISBN 978-x-xxxx-xxxx-x/YY/MM...\$15.00

<https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 INTRODUCTION

Large Language Models (LLMs) like Transformers rely on intensive matrix multiplications for their self-attention and feed-forward blocks. As shown in the Figure 1, computing the Q, K, V, and output linear projections during Multi-Head Self-Attention in each Transformer layer entails multiplying large matrices (input activation $\times W_q/W_k/W_v/W_o$). DistilBERT [5], a compact Transformer variant [4], still performs billions of multiply-accumulate operations per inference. These operations are memory-bandwidth heavy on general processors, motivating specialized hardware acceleration. FPGAs offer custom parallelism and energy efficiency for matrix math, but deploying LLM acceleration on edge FPGAs presents challenges due to limited resources and power.

Motivated by the observation that the Q, K, and V projections form a significant computational bottleneck, we strategically concentrated our efforts on accelerating these key components. This targeted approach allowed us to fully optimize our design for the resource constraints of the Xilinx Kria KV260 Vision AI Kit (housing a Zynq UltraScale+ FPGA), while still delivering substantial performance improvements. Our approach exploits data reuse and parallelism to speed up matrix multiplication operations within the tight DSP and BRAM budget of an edge FPGA. We adopt a block/tiled matrix multiply approach: one input matrix (“A”) remains in on-chip memory, while the other (“B”) is processed in smaller column blocks to maximize reuse. Unlike prior LLM accelerators that apply model-wide optimizations (e.g., pruning, specialized sparse formats), our work directly accelerates dense GEMM (General Matrix Multiply) cores within the Q, K, V projection operations. To encourage reproducibility and further research, we provide our FPGA design, software interface, and benchmarking scripts in an open-source repository at **GitHub**.

2 CONTRIBUTIONS

In this work, we present a specialized FPGA-based accelerator for key matrix multiplication operations in Transformer self-attention. While elements of our contributions are woven throughout the *Introduction* and *Discussion* sections, here we explicitly summarize the main novel aspects and potential benefits for future research:

- **Targeted Focus on Self-Attention Bottleneck:** We pinpoint the Q, K, and V projections within Multi-Head Self-Attention as critical performance bottlenecks. Unlike many existing FPGA accelerators that aim to cover a broad range of model operations (often at the cost of higher resource

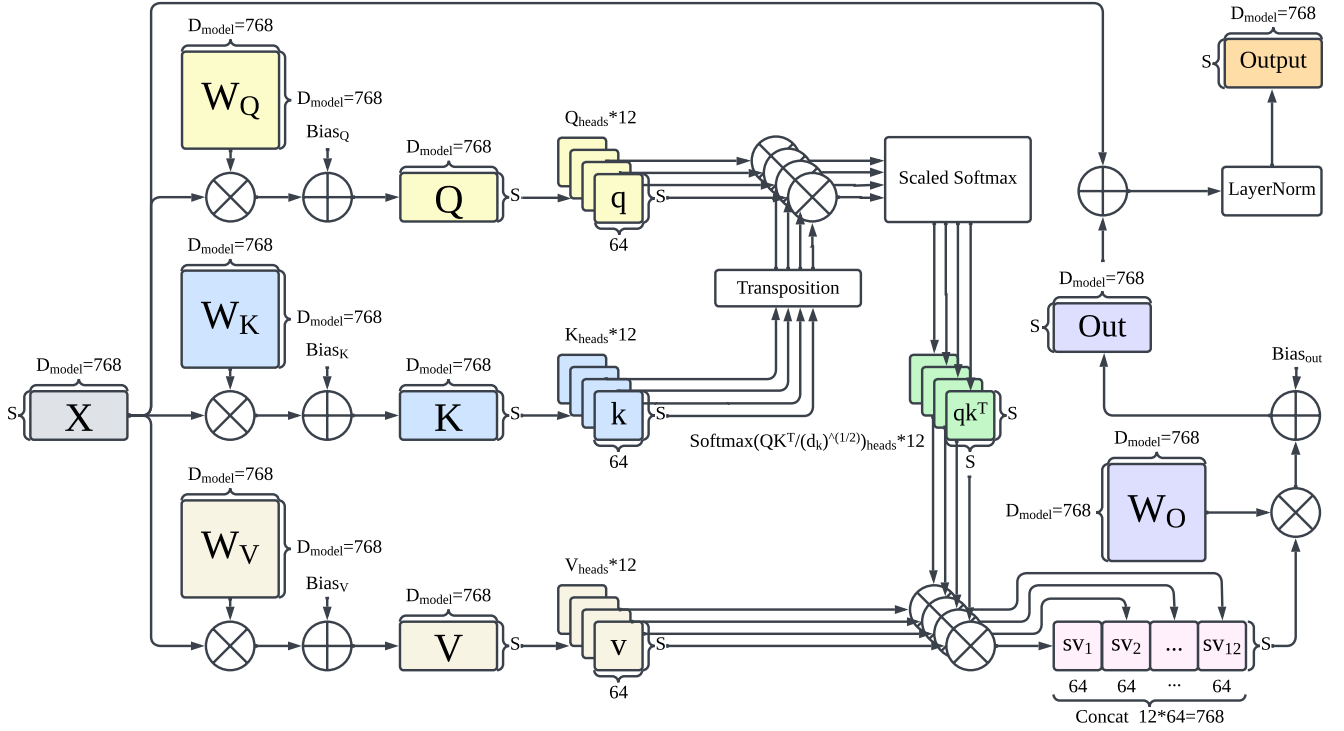


Figure 1: MHA Pipeline in transformer

demands), our design is tightly focused on these fundamental matrix multiplications, making it more suitable for edge FPGAs with constrained resources.

- **Tiled and Data-Centric Design:** By employing two-level tiling and on-chip data persistence, we minimize off-chip memory transfers and fully leverage data reuse. Our systolic-like unrolled compute engine—sized specifically to fit KV260 resources—enables high throughput (up to 3.12 GFLOPs) while staying within strict DSP and BRAM budgets. This data-centric approach is amenable to analytic frameworks such as MAESTRO [2] and other DNN dataflow studies [3].
- **High-Level Synthesis (HLS) and Easy Integration:** Implementing the accelerator in C++ HLS facilitates rapid design space exploration and more accessible customization. The accelerator is integrated via AXI4 and PYNQ overlays, offering a user-friendly interface that can be extended to other matrix-multiply-based workloads with minimal effort.
- **Quantized DistilBERT Integration:** We replace standard PyTorch Q, K, and V linear layers with our FPGAQuantizedLinear module and validate end-to-end functionality on a quantized DistilBERT. This demonstrates near-lossless accuracy while providing up to a 7× speedup over PyTorch CPU baselines for these layers. Our open-source release includes scripts for quantization, FPGA invocation, and reproducible benchmarking, potentially lowering the barrier for researchers to adopt FPGA acceleration in transformer applications.

- **Roadmap for Future Scaling:** Although we focus on moderate matrix sizes (e.g., 64×768, 768×3072) representative of DistilBERT, our tiled architecture offers a blueprint for scaling to larger transformer variants. The clear separation of on-chip buffering, tiling logic, and computation modules can be extended to handle more extensive hidden dimensions and different attention heads with minimal redesign.

By addressing a critical compute bottleneck within Transformers and providing an open-source HLS-based implementation, we aim to advance both the practical adoption of FPGAs in edge-deployed large language models and future studies on optimized dataflows for deep learning inference.

3 RELATED WORK

FPGA Accelerators for Transformers

Recent research has produced FPGA-based LLM accelerators such as FlightLLM [6] and SSR [7]. **FlightLLM** (FPGA'24) maps entire LLM inference flows onto FPGAs, leveraging sparsity and mixed-precision; implemented on a high-end Alveo U280, it achieves **6× higher energy efficiency** than an NVIDIA V100 GPU [6] by using sparse DSP chains and always-on-chip decoding. **SSR** (FPGA'24) explores launching multiple accelerators in parallel versus sequentially to balance latency and throughput. On a Versal ACAP VCK190, SSR attains up to **2.5× throughput** versus an NVIDIA A10G GPU, with **8.5× energy efficiency** [7]. **FAMOUS** focuses on the Transformer's attention mechanism: a flexible multi-head attention core on Alveo U55C that sustains 328 GOPS and runs **2.6× faster than**

an NVIDIA V100 GPU and $1.3\times$ faster than prior FPGA designs. Earlier, Qi *et al.* (GLSVLSI'21) combined model compression (pruning) with FPGA optimization to fit Transformers onto FPGAs. These works, however, target large FPGAs or datacenter contexts and often integrate advanced techniques (sparsity, multi-engine concurrency) less applicable to small-edge devices.

In addition to large-scale accelerators, recent research has advanced data-centric analytical frameworks to understand data reuse and mapping strategies in DNN accelerators. For instance, MAESTRO [2] and the approach presented in "Understanding Reuse, Performance, and Hardware Cost of DNN Dataflows: A Data-Centric Approach" [3] provide comprehensive models for quantifying data movement, reuse, and the associated hardware costs. Although these frameworks target a broader range of DNN workloads, they share foundational principles with our work—particularly the emphasis on maximizing both temporal and spatial data reuse—which informs our targeted accelerator design for Transformer self-attention on edge FPGAs.

Our Approach vs Prior Art

Unlike FlightLLM or SSR, which assume abundant resources (HBM memory, > 9000 DSPs) and apply complex optimizations, our design emphasizes **dense GEMM acceleration on a resource-constrained FPGA**. We focus on core matrix-multiply throughput via tiling and on-chip buffering, rather than algorithmic sparsification. In contrast to FAMOUS's multi-head pipeline on big FPGAs, we implement a single GEMM core optimized for reuse and run it at 100 MHz on the KV260 (which has far fewer BRAM/DSP). Recent work on complete Transformer accelerators, such as the Hardware Accelerator for Multi-Head Attention and Position-Wise Feed-Forward in the Transformer [1], also highlights the importance of efficient dataflow management. However, by concentrating on an efficient memory hierarchy and loop-level optimizations specifically for the Q, K, and V projections, our design is uniquely tailored for edge deployment scenarios.

4 SYSTEM DESIGN

The accelerator is based on a **tiled matrix multiplication architecture** tailored for multiplying matrices of size 64×768 (A) with 768×3072 (B), matching the dimensions used in DistilBERT. By decomposing large matrices into smaller tiles that can be loaded into fast on-chip memory (BRAM), the design improves data locality, reduces off-chip DRAM accesses, and enables extensive parallelism. Figure 2 illustrates an overview of the tiling logic.

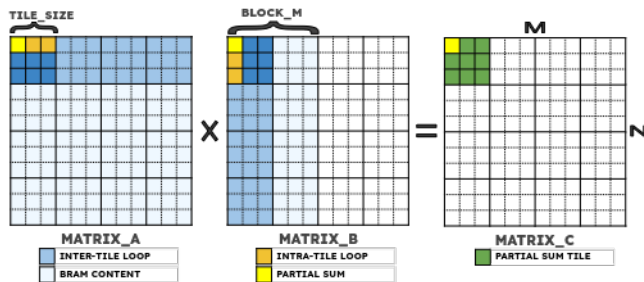


Figure 2: Overview of tiled matrix multiplication approach.

High-Level Dataflow Mapping Strategies

Our design leverages multi-level mapping strategies to optimize both computation and data movement:

I. Temporal Mapping:

- **Tiling:** We employ a two-level tiling approach (Block Tiling with $M_b = 256$ and Inner Tiling with $T = 32$) to decompose large matrices into smaller tiles. This allows matrix A to be loaded once into on-chip BRAM, significantly reducing DRAM accesses.
- **Pipelining:** The accumulation loop over the K dimension is pipelined ($\Pi=1$), ensuring continuous processing and high throughput.
- **Data Persistence:** Persistent storage of matrix A enhances data reuse across multiple computations.

II. Spatial Mapping:

- **Loop Unrolling:** Fully unrolling the innermost loops creates a 32×32 array of multipliers-adders for parallel computation.
- **Memory Partitioning:** Partitioning local tile arrays into registers enables simultaneous data access, reducing memory access bottlenecks.

III. Inter/Intra-tile Mapping:

- **Inter-tile:** Matrix B is partitioned into 256-column blocks, with each block assigned to a different processing column-tile with corresponding row-tile from Matrix A for parallel execution.
- **Intra-tile:** Within each tile, pipelined and fully unrolled loops enable efficient parallel processing of tiles.

Together, these mapping strategies optimize the balance between computation and data movement, enabling high throughput on the resource-constrained Xilinx KV260.

Recent data-centric approaches, such as those in MAESTRO [2] and the work on DNN dataflows [3], provide an analytical perspective on how temporal and spatial mapping can be exploited to maximize data reuse. Inspired by these frameworks, our design explicitly separates temporal mapping—through techniques like tiling, pipelining, and data persistence—from spatial mapping—through loop unrolling and memory partitioning. This dual strategy minimizes off-chip accesses and maximizes parallelism, thereby achieving high efficiency on an edge FPGA platform.

4.1 HLS Implementation and Parallel Computation

At the implementation level, the high-level mapping strategies are realized using Vivado HLS directives:

- **Unified Tiling and Memory Hierarchy:** Matrix A is loaded once into persistent BRAM, while matrix B is streamed in blocks. Local buffers (register-level tiling) store 32×32 tiles of A and B for rapid access.
- **Parallel Computation:** The innermost loops are fully unrolled, creating a 32×32 array of multiplier-adder units that operate concurrently. The accumulation loop over the K dimension is pipelined with an initiation interval (Π) of 1, achieving up to $1024 \text{ int8}\times \text{int8}$ MAC operations per clock cycle.

- **Practical HLS Optimization:** Loop pipelining, unrolling, and array partitioning work in tandem to implement the temporal and spatial mapping strategies in hardware, thereby maximizing parallelism while respecting the FPGA's resource constraints.

4.2 AXI Interface and Integration

The accelerator is implemented as an IP core and integrated into the system via AXI SmartConnect interfaces, enabling communication with the Zynq UltraScale+ MPSoC. As shown in Figure 3, for memory and control, three AXI4 master ports (one each for input A, input B, and output C) stream data to/from external DDR, and an AXI4-Lite slave interface allows the host (CPU) to configure dimensions and start the kernel [8]. The design supports a special control flag `update_A` so that the host can choose to reuse the last loaded A matrix for subsequent calls [10] – useful when processing multiple B batches with the same weights (e.g., iterating over attention heads or sequences). On the KV260, the accelerator IP is integrated into the FPGA fabric and invoked from software via PYNQ drivers.

5 IMPLEMENTATION

Algorithm 1 High-Level Tiled Matrix Multiplication Accelerator

Require: Matrices $A (N \times K)$, $B (K \times M)$, output matrix $C (N \times M)$, flag `update_A`; constants: `BLOCK_M`, `TILE_SIZE`

Ensure: $C \leftarrow A \times B$

```

1: if update_A is true then
2:   Copy A from DDR into persistent on-chip BRAM
3: end if
4: for each column block  $j\_block$  in  $[0, M)$  with step BLOCK_M do
5:    $current\_block\_M \leftarrow \min(BLOCK\_M, M - j\_block)$ 
6:   Load block of B into on-chip BRAM
7:   for each tile row  $i_0$  in  $[0, N)$  with step TILE_SIZE do
8:     for each tile column  $j_0$  in  $[0, current\_block\_M)$  with
       step TILE_SIZE do
9:       Initialize local output tile  $localC \leftarrow 0$ 
10:      for each tile  $k_0$  in  $[0, K)$  with step TILE_SIZE do
11:        Load  $localA$  and  $localB$  tile from Bram
12:         $localC \leftarrow localC + localA \times localB$ 
13:      end for
14:      Write tile  $localC$  to C at the appropriate offset
15:    end for
16:  end for
17: end for

```

We developed the accelerator in C++ HLS (Xilinx Vitis HLS). The HLS code explicitly declares the top-level loops and applies pragmas for pipelining and unrolling. For example, the innermost multiply-accumulate loops are annotated with `#pragma HLS UNROLL` (for the `ii` and `jj` loops over the tile) and the `k`-loop with `#pragma HLS PIPELINE II=1`. Similarly, we partition the local tile arrays into registers (`#pragma HLS ARRAY_PARTITION complete`), so each element can be accessed in parallel without memory banking conflicts.

Tile size selection. We experimented with several tile sizes (e.g., $T = 16$, $T = 64$) and settled on $T = 32$ based on routing feasibility, timing closure, and performance. Larger tiles such as $T = 64$ improved theoretical parallelism but made place-and-route challenging at 100 MHz. Smaller tiles (e.g., $T = 16$) simplified timing but reduced total throughput. Hence, $T = 32$ emerged as an optimal balance between resource usage and achievable frequency.

Handling partial tiles. In real-world cases, N , K , or M may not be perfectly divisible by 32. We use boundary checks within our tiled loops to handle leftover rows and columns, loading and computing smaller sub-tiles. In our experiments, overhead for these partial tiles is minimal (~1–2% time difference), indicating our design scales well to arbitrary matrix dimensions.

The accelerator IP was compiled to RTL with Vivado HLS and integrated in Vivado along with AXI SmartConnect for data movement. We ran at a conservative 100 MHz PL clock, easily meeting timing closure. During synthesis, some `int8` multiplications were mapped to LUTs once available DSP blocks were exhausted, reflecting the high level of parallelism (up to 1024 concurrent MACs).

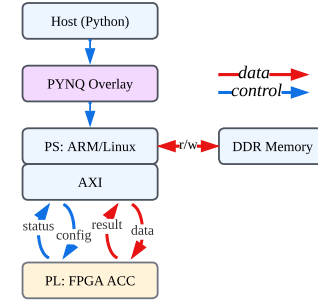


Figure 4: Diagram of the PYNQ-based interface on the KV260.

On the software side, we created a PYNQ overlay in Figure 4 to manage buffers and FPGA invocation. The host (Python) configures the accelerator via the PYNQ overlay and the Processing System (PS), as depicted in the diagram of the PYNQ-based interface on the KV260—where the AXI interface provides bidirectional communication (Config/Status, Result) between the PS and the FPGA Accelerator, and DDR memory holds buffers for A, B, and C. Specifically, the host allocates contiguous buffers for A, B, and C (using `pynq.allocate`), sets up the accelerator registers (pointing to physical addresses of buffers and dimensions N , K , M), and toggles `AP_START` [10]. We wrapped this in a Python `call_fpga()` function that optionally retains A between calls. This allowed integration with a quantized DistilBERT: we replaced the PyTorch linear layers for Q, K, V in attention with calls to the FPGA (using custom PyTorch extensions to invoke `call_fpga()`), feeding `int8` quantized weights and inputs. In doing so, we ensured consistent scaling by using symmetric quantization with a fixed scale factor and zero-point for the weights and activations so that the `int8` results could be dequantized to match the FP32 baseline. We verified that the FPGA outputs matched CPU computation exactly for small test matrices and remained within quantization error for end-to-end model outputs.

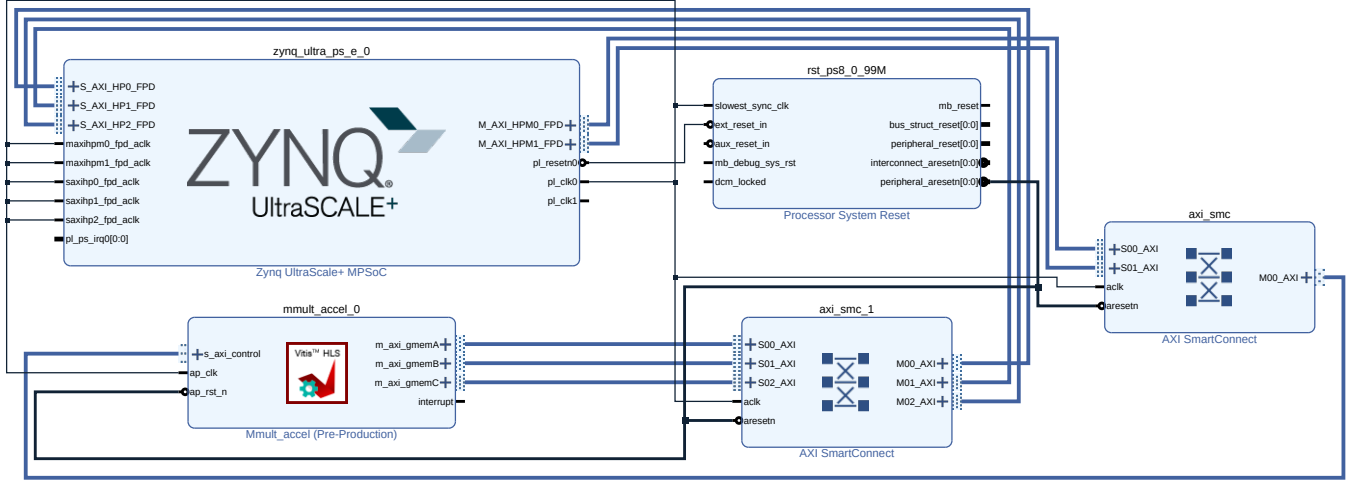


Figure 3: Vivado Block Design

6 PERFORMANCE EVALUATION

6.1 Experimental Setup

We benchmarked the accelerator on the KV260 board running Ubuntu with PYNQ support. The host’s CPU consists of 4× Arm Cortex-A53 @ 1.5GHz and 2× Cortex-R5F @ 600MHz, which we used for running baseline software matrix multiplication (NumPy and PyTorch) and orchestrating FPGA execution.

Power measurement methodology. We used the KV260’s on-board power sensors to obtain instantaneous readings every 50–100 ms during execution. Energy consumption was approximated by integrating (summing) the power samples over the total runtime of the matrix multiplication. We then compared this to the CPU baseline, similarly sampling the system power with the FPGA accelerator quiescent.

Latency was measured using Python’s `time()` function along with the accelerator’s built-in timers.

Table 1: Resource Utilization for Tiled MatMul Accelerator on XCK26 (KV260) at 100 MHz

Resource	Used	Available	Utilization
BRAM	126	144	88%
DSP48E	1040	1248	83%
FF	102741	237600	43%
LUT	71050	118800	60%

6.2 Benchmark Cases

We evaluated two scenarios:

- (1) **Standalone GEMM:** Random matrices of varying sizes were used to assess raw computational performance. We conducted tests on two representative cases:
 - A DistilBERT attention case of $(64, 768) \times (768, 768)$, corresponding to approximately 37.7 million MACs.
 - A Feed-Forward Network (FFN) case of $(64, 768) \times (768, 3072)$, corresponding to roughly 150 million MACs.

- (2) **DistilBERT Attention Throughput:** Here, we integrated the FPGA accelerator into a quantized DistilBERT model by replacing the standard Q, K, and V linear layers with our custom `FPGAQuantizedLinear` module. This module offloads only the critical Q, K, V matrix multiplications from the Multi-Head Self-Attention (MHA) block.

Framework	Latency (s)	Throughput (GFLOPs)
NumPy (ARM CPU)	20.72	0.01
PyTorch (ARM CPU)	0.67	0.45
FPGA (compute)	0.09	3.12
FPGA (end-to-end)	0.11	2.85

Table 2: Performance on a 768×3072 matrix multiplication.

For the 768×3072 multiplication, the FPGA achieved a latency of 9.67 ms compared to 67.84 ms on PyTorch (ARM) and 2072.25 ms on NumPy (without optimized BLAS), yielding a **7.0× speedup** over PyTorch and a **214× speedup** over NumPy. The effective throughput was approximately 3.12 GFLOPs for the core computation, slightly reducing to 2.85 GFLOPs when including data transfer overhead.

Energy measurements further underscored the benefits of our approach: the KV260 consumed about 3.3 W during FPGA operation compared to a baseline of 3 W in idle mode, while the ARM core used roughly 3.2 W when running PyTorch. This resulted in an energy efficiency of approximately 0.5 J per 768×3072 multiply on the FPGA versus 2 J on the ARM—a roughly 4× improvement.

In the **DistilBERT Attention** scenario, our focus was on accelerating the Q, K, and V linear projections. The `FPGAQuantizedLinear` module performs the following:

- Quantizes input activations and weights to int8.
- Converts these arrays into PYNQ buffers for efficient data transfer.
- Offloads the core 2D matrix multiplication to the FPGA accelerator.

- Dequantizes the resulting int32 outputs back to floating point and adds bias if necessary.

Benchmarking compared the inference times of a CPU-only forward pass against those with FPGA acceleration for these projections. Our experimental results show that while a CPU-only forward pass required approximately 1.14 seconds, offloading the Q, K, and V projections reduced the compute time for matrix multiplications to about 0.43 seconds. Consequently, the overall end-to-end speedup for DistilBERT inference is around $2\times$. Although the transmission latency introduced by the PYNQ overlay tempers the speedup relative to the standalone GEMM performance, the nearly identical prediction confidence levels (e.g., 99.95% on CPU versus 99.80% on FPGA) decisively demonstrate that our approach robustly accelerates a critical component of the attention mechanism without compromising accuracy.

7 CHALLENGES AND OPTIMIZATIONS

During development, we encountered several bottlenecks and addressed them via optimized HLS transformations:

- **Memory Bandwidth Bottleneck:** Initially, reading A and B from DRAM each time was a limiting factor. We introduced **persistent BRAM buffers** for A and block BRAM for B to cut down redundant transfers [8]. Additionally, we used wide AXI bursts to transfer blocks of B efficiently (256 columns at a time). The `update_A` flag allows us to amortize the cost of loading A when it remains constant.
- **HLS Pipeline Timing:** Achieving $II=1$ in the inner loop was challenging given 1024 operations in parallel. We ensured no loop-carried dependencies by fully unrolling the independent MAC operations. The HLS tool initially issued warnings about DSP utilization; *we mitigated this by allowing some multipliers to be mapped to LUTs once DSPs were fully consumed*, which still met timing requirements.
- **Loop Bounds and Edge Conditions:** We implemented tile loops to cover matrix sizes not multiples of 32 (with boundary checks inside the tile loads and compute). *Testing revealed that partial-tile overhead was low, so no specialized hardware was needed for fractional tiles.*
- **Persistent On-Chip Storage:** Storing the entire A (64×768 int8) in BRAM consumed about 48 KB, which was acceptable. However, storing a full 768×768 B (approximately 589 KB) was impossible, hence the block tiling. We tuned `BLOCK_M=256` after evaluating BRAM usage.
- **Quantization:** Converting DistilBERT weights and activations to int8 required careful calibration. We used PyTorch's static quantization to get int8 weights for the Q, K, V linear layers. Our quantized accelerator produced negligible accuracy loss ($<0.5\%$ deviation in attention outputs).
- **Clock Frequency vs Parallelism Tradeoff:** *We attempted tile sizes of $T=16$ and $T=64$ to explore the design space. $T=16$ reduced concurrency (fewer multipliers in parallel), lowering throughput. $T=64$ improved concurrency but complicated place-and-route, failing timing closure at 100 MHz. Thus, $T=32$ offered a well-balanced solution.*

8 DISCUSSION AND FUTURE WORK

The project demonstrates that a relatively small FPGA can accelerate key operations of Transformer models. By deliberately focusing on the acceleration of the Q, K, and V linear projections within the Multi-Head Self-Attention module, we have optimized a critical performance bottleneck under resource-constrained conditions. The substantial improvements observed in standalone GEMM benchmarks indicate that extending this targeted approach to other components could yield even more significant end-to-end gains. Our results echo the insights from broader data-centric frameworks [2, 3] that emphasize the benefits of maximizing temporal and spatial data reuse.

- **Focused Acceleration:** Our accelerator currently targets the Q, K, and V projections—a deliberate choice that allowed for deep optimization and efficient use of resources. Future work can build on this foundation by integrating additional components, such as softmax operations and the Feed-Forward Network (FFN) layers.
- **Scaling to Larger Models:** Our design currently targets matrices with input activation sizes up to 64×768 . For larger Transformer models (e.g., with hidden sizes of 1024 or 4096), advanced memory management strategies—such as double-buffered streaming and ping-pong buffering—will be essential to hide latency and effectively manage data flow.
- **Reducing Data Transfer Overhead:** To unlock the FPGA's full potential, future work should further minimize data movement between the CPU and FPGA. Leveraging more persistent on-chip memory for activations and implementing DMA could significantly reduce transfer latency.
- **Improving CPU-FPGA Execution Pipelining:** Maximizing performance requires overlapping CPU and FPGA execution. While the FPGA effectively accelerates the QKV projections, concurrent processing of subsequent stages (e.g., attention softmax or FFN layers) on the CPU is essential for further throughput gains. Techniques such as double buffering can facilitate simultaneous data transfer and computation, thereby reducing idle time. Addressing the unignorable transmission delays associated with the current PYNQ overlay and custom forward pass represents a promising direction for future work.
- **Optimizing FPGA Utilization and Scaling:** Future improvements might include parallelizing the Q, K, and V projections on separate systolic arrays, as well as keeping the FPGA accelerator active across multiple layers rather than resetting registers and flushing memory for each new computation. Such strategies would reduce overhead and further improve efficiency.

9 CONCLUSION

We have presented the design and implementation of an FPGA-Based Tiled Matrix Multiplication Accelerator for Transformer Self-Attention on the Xilinx KV260 SoM. Our work demonstrates that through loop tiling, on-chip buffering, and parallel unrolling, even a resource-constrained FPGA can significantly accelerate the matrix multiplication operations that underpin Transformer attention mechanisms. Standalone GEMM benchmarks and the acceleration

of Q, K, and V projections achieved a notable $7\times$ speedup over PyTorch CPU execution and marked improvements in energy efficiency.

While the overall end-to-end DistilBERT performance currently exhibits a $2\times$ speedup, this result is largely influenced by the transmission delays inherent to the current PYNQ overlay and custom forward pass. These unignorable overheads clearly indicate an opportunity for further optimization. Future work focused on reducing data transfer latencies and integrating additional components—such as softmax and FFN layers—along with enhanced system-level optimizations is expected to yield even greater performance gains. In summary, this project not only confirms the viability of FPGA-based acceleration for key Transformer operations but also establishes a strong foundation for future advancements in efficient, scalable deep learning inference on power-constrained edge devices.

ACKNOWLEDGMENTS

The authors would like to thank Professor Sitao Huang from the University of California, Irvine, for his invaluable guidance and insightful discussions throughout this project. Special thanks are also extended to Sicheng Chen for his excellent collaboration.

REFERENCES

- [1] Siyuan Lu et al. 2020. *Hardware Accelerator for Multi-Head Attention and Position-Wise Feed-Forward in the Transformer*. arXiv:2009.08605. <https://arxiv.org/abs/2009.08605>.
- [2] Hyoukjun Kwon, Prasanth Chatarasi, Vivek Sarkar, and Tushar Krishna. 2020. *MAESTRO: A Data-Centric Approach to Understand Reuse, Performance, and Hardware Cost of DNN Mappings*. IEEE Micro, 40(3): xx–yy, 2020. <https://doi.org/10.1109/MM.2020.2985963>.
- [3] Hyoukjun Kwon, Prasanth Chatarasi, Michael Pellauer, Angshuman Parashar, Vivek Sarkar, and Tushar Krishna. 2019. *Understanding Reuse, Performance, and Hardware Cost of DNN Dataflows: A Data-Centric Approach*. In MICRO-52, Oct. 12–16, 2019, Columbus, OH, USA. <https://doi.org/10.1145/3352460.3358252>.
- [4] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. 2017. *Attention Is All You Need*. In Proc. of Advances in Neural Information Processing Systems (NeurIPS) 30. <https://arxiv.org/abs/1706.03762>
- [5] Victor Sanh, Lysandre Debut, Julien Chaumond, and Thomas Wolf. 2019. *DistilBERT, a distilled version of BERT*. arXiv:1910.01108. <https://arxiv.org/abs/1910.01108>
- [6] Shulin Zeng et al. 2024. *FlightLLM: Efficient Large Language Model Inference with a Complete Mapping Flow on FPGAs*. In Proceedings of FPGA '24.
- [7] Jinming Zhuang et al. 2024. *SSR: Spatial Sequential Hybrid Architecture for Latency-Throughput Tradeoff in Transformer Acceleration*. In Proceedings of FPGA '24.
- [8] HLS Design Code. https://github.com/Richielee630/MatMul_SA.git.
- [9] SAMA PYNQ Benchmark Results. Available at: <https://github.com/Richielee630/TMMA/tree/main/pynq>.
- [10] Quantized DistilBERT Forward Pass. Available at: <https://github.com/Richielee630/TMMA/tree/main/pynq>.